

Python Fundamentals

Sessions 1–3 · Data Types, Literals, Operators, Loops & Strings

Study Notes

Compiled 14 June 2026

Contents

Orientation	6
The <code>print()</code> Function	7
Printing single values	7
Printing several values at once	7
The <code>sep</code> argument — controlling the separator	7
The <code>end</code> argument — controlling the line ending	7
Data Types	8
Numeric types	8
Integer (<code>int</code>)	8
Float	9
Boolean (<code>bool</code>)	9
Complex	9
Collection types	9
List — an ordered, mutable sequence	9
Tuple — an ordered, immutable sequence	9
Set — an unordered collection of unique elements	9
Dictionary — key–value pairs	9
Variables and Binding	10
Dynamic typing	10
Dynamic binding	10
The <code>None</code> keyword	11
Identifiers must exist before use	11
Taking User Input	12
<code>input()</code> always returns a string	13
Type Conversion	13
Why conversion is needed: the <code>int + str</code> error	13
Explicit conversion truncates toward zero (float → int)	14
Converting input before the operation	14
Converting at the point of input	14
Implicit conversion: booleans behave as numbers	15
Literals	15

Integer literals in different bases	15
Float literals and scientific notation	16
Complex literals	16
String literals	17
Boolean and <code>None</code> literals	17
Operators	18
Arithmetic operators	18
Relational operators	19
Logical operators	19
Bitwise operators	19
Assignment operators	20
Membership operators	21
Control Flow: Loops	21
The <code>for</code> loop and <code>range()</code>	21
Iterating directly over a sequence	22
The <code>for...else</code> and <code>while...else</code> construct	23
<code>while...else</code>	24
Worked example: printing primes with <code>for...else</code>	24
Strings	25
What a string is	25
Creating strings	26
Quotes inside strings	26
Accessing characters: indexing	26
Slicing	27
Basic slice	27
Omitting an endpoint	27
Slicing with a step	27
Negative step — walking backwards	28
Reversing a string	28
Slicing with negative indices	28
Strings are immutable	29
Deleting strings	29
Operations on strings	30
Arithmetic operations	30
Relational operations	30
Negative step — walking backwards	31
Reversing a string	31
Slicing with negative indices	32
Strings are immutable	32
Deleting strings	33
Operations on strings	33

Arithmetic operations	33
Relational operations	34
Logical operations	34
Loops on strings	35
Membership operations	35
String functions	35
Common functions: len, max, min, sorted	35
Capitalize, Title, Upper, Lower, Swapcase	36
Count, Find, Index	36
Endswith and Startswith	37
String Formatting	37
String Validation Methods	37
Split and Join	38
Replace	38
Strip	38
8. What Does the 'is' Operator Do?	42
9. Disadvantages of Python.	42
10. How strings are stored in Python?	43
11. What is the Zen of Python?	43
12. Identity operator (is) vs Equality operator (==)?	43
13. Modules vs Packages vs Library in Python	43
14. Why 0.3 - 0.2 is not equal to 0.1 in Python?	43
15. What are Python Docstrings?	43
Lists in Python	44
What are Lists?	44
Array vs. Lists	44
How Lists are Stored in Memory	44
How Dynamic Arrays Work	45
Characteristics of a List	45
Creating a List	45
Accessing Items from a List	46
Editing Items in a List	47
Deleting Items from a List	47
Membership Operations	48
Loops	48
List Methods	48
List Comprehension	49
Traversing a List	51
Zip Function	51
Disadvantages of Python Lists	51
Tuples	52

Characteristics of Tuples	52
Plan of Attack	52
Creating Tuples	52
Accessing Items	53
Editing Items	53
Adding Items	53
Deleting Items	53
Operations on Tuples	53
Difference between Lists and Tuples	54
Immutability and Aliasing	54
Special Syntax	55
Tuple Unpacking	55
Zipping Tuples	55
Sets	56
Introduction to Sets	56
Characteristics	56
Creating Sets	56
Accessing Items	57
Adding Items	57
add()	57
update()	57
Deleting Items	57
discard(item)	58
remove(item)	58
pop()	58
clear()	58
del	58
Set Operations	58
Membership Test	59
Iteration	59
Set Functions	59
Other Set Methods	59
union() and update()	59
intersection() and intersection_update()	59
difference() and difference_update()	59
symmetric_difference() and symmetric_difference_update()	60
Checking Set Relations	60
copy()	60
Frozenset	60
Creating Frozenset	60
What works and what does not	60

When to use Frozenset?	60
Set Comprehension	61
Dictionaries	61
Introduction to Dictionaries	61
Characteristics	61
Accessing Items	62
Adding Key-Value Pair	63
Editing Key-Value Pair	63
Removing Key-Value Pair	63
Dictionary Operations	64
Membership	64
Iteration	64
Dictionary Functions	64
len()	64
sorted()	64
max() / min()	65
items(), keys(), values()	65
Dictionary Comprehension	65
Functions	66
Introduction to Functions	66
Types of Arguments	67
*args and **kwargs	67
*args	67
**kwargs	68
Accessing Docstrings	68
Points to remember while using *args and **kwargs	68
How Functions are Executed in Memory	68
Variable Scope	69
Global vs. Local Variables	70
UnboundLocalError	70
The global Keyword	70
Modifying Mutable Global Objects	71
The “Sticky Note” Rule (Immutability)	71
Nested Functions	72
Functions as First-Class Citizens	72
Lambda Functions	73
Higher Order Functions (HOF)	74
1. map()	74
2. filter()	75
3. reduce()	75
Memory and Object References	76

Memory Models	76
Aliasing	76
Garbage Collection	76
Mutability and Its Dangers	77
Cloning (Shallow vs Deep Copies)	77
Shallow Copy	77
Deep Copy	77
Memory Layout of Types	77
Nested Lists	77
Strings	77
Tuples vs. Lists in Memory	77
Sets and Dictionaries	78
Object-Oriented Programming (OOP) Concepts	78
Class and Object	78
The Four Pillars of OOP	79
File Handling	79
Opening and Closing Files	79
File Modes	79
Overwriting vs. Appending	80
Reading from Files	80
<code>read()</code>	80
<code>readline()</code>	80
<code>writelines()</code>	81
The Context Manager (<code>with</code> block)	81
Reading Large Files in Chunks	81

Orientation

These notes cover the opening three sessions of an introductory Python course: the building blocks of the language as demonstrated live in Google Colab notebooks. The material moves from printing and data types, through variables, user input, type conversion, literals and operators, into control flow with loops, and finally into strings.

Everything here is shown by running small snippets in a notebook cell and reading the result. Code is reproduced exactly as executed; outputs are the values Python actually produces. Where a captured cell displays a result that disagrees with its code (a common notebook hazard), the discrepancy is called out rather than copied blindly.

A recurring theme worth holding in mind from the start: **Python is dynamically typed**, every value has a type, and many “errors” beginners hit are really the language being strict about what operations a given type supports.

The `print()` Function

Prerequisites Block: * **Assumed Knowledge:** None. This is the very first concept.

First-use gloss: `print()` is a built-in function that writes its arguments to standard output. It accepts any number of values and, by default, separates them with a single space and ends with a newline.

Printing single values

```
print(7)
print(7.7)
print(True)
```

```
7
7.7
True
```

A whole number, a decimal, and a boolean each print in their natural textual form.

Printing several values at once

Passing multiple arguments prints them on one line, space-separated:

```
print('Hello', 1, 4.5, True)
```

```
Hello 1 4.5 True
```

The arguments may be of mixed types — `print()` converts each to text independently.

The `sep` argument — controlling the separator

`sep` replaces the default single space placed *between* arguments:

```
print('Hello', 1, 4.5, True, sep='/')
```

```
Hello/1/4.5/True
```

The `end` argument — controlling the line ending

By default every `print()` finishes with a newline, so consecutive calls stack vertically:

```
print('hello')
print('world')
```

```
hello
world
```

Setting `end` overrides that trailing newline. Here both calls share a line, joined by the chosen string:

```
print('hello', end='-')
print('world')
```

```
hello-world
```

Key point. `sep` controls what goes *between* the arguments of a single `print()`; `end` controls what is appended *after* the last one. The default values are `sep=' '` and `end='\n'`.

Data Types

Python ships with a set of built-in types. The snippets below print one literal of each so its display form is visible.

Numeric types

Integer (`int`)

Python integers have **arbitrary precision** — they are limited only by available memory, not by a fixed width.

```
# Integer
print(8)
# 1*10**308
print(1e309)
```

```
8
inf
```

The second line is instructive. `1e309` is written as a *float* literal, and `10**309` exceeds the range a floating-point number can represent, so Python yields the special floating-point value `inf` (infinity) rather than a number. This overflow behaviour belongs to floats; the

unbounded `int` type would have held the exact value had it been written as an integer expression.

Float

Decimal numbers are of type `float`. They are demonstrated further under [Literals](#).

Boolean (`bool`)

`True` and `False`. Booleans are a subtype of integers in Python, a fact that surfaces directly under [Type Conversion](#).

Complex

Numbers with a real and imaginary part, written with a `j` suffix on the imaginary component (see [Literals](#)).

Collection types

List — an ordered, mutable sequence

```
# List -> C -> Array
print([1, 2, 3, 4, 5])
```

```
[1, 2, 3, 4, 5]
```

A list is Python's closest analogue to the array of a language like C, but it is dynamic and may hold mixed types.

Tuple — an ordered, immutable sequence

```
# Tuple
print((1, 2, 3, 4, 5))
```

```
(1, 2, 3, 4, 5)
```

A tuple looks like a list but uses parentheses and cannot be changed after creation.

Set — an unordered collection of unique elements

```
# Sets
print({1, 2, 3, 4, 5})
```

A set is written with braces and stores no duplicates and no order.

Dictionary — key-value pairs

```
# Dictionary
print({'name': 'Nitish', 'gender': 'Male', 'weight': 70})
```

```
{'name': 'Nitish', 'gender': 'Male', 'weight': 70}
```

A dictionary maps keys to values, also written with braces but using `key: value` pairs.

First-use gloss: *List* `[ ]` — an ordered, changeable collection of items. **First-use gloss:** *Tuple* `( )` — an ordered, fixed collection of items. **First-use gloss:** *Set* `{ }` — an unordered collection of unique items. **First-use gloss:** *Dictionary* `{k: v}` — a collection of keyed lookups. Braces serve both sets and dictionaries; the presence of `key: value` pairs distinguishes a dictionary.

Variables and Binding

Prerequisites Block: * **Assumed Knowledge:** Basic data types (integers, floats, booleans, strings).

First-use gloss: A **variable** is a name bound to a value. Assignment uses `=`.

Dynamic typing

Python is **dynamically typed**: a variable's type is inferred from the value assigned to it, and is never declared.

```
# Dynamic Typing
a = 5

# Static Typing
int a = 5
```

The first form is valid Python. The second (`int a = 5`) is how a *statically typed* language such as C or Java would force you to declare the type up front — it is shown only as a contrast and is **not** valid Python syntax.

Dynamic binding

Heavy. Variables in Python are fundamentally different from variables in languages like C or Java.

Analogy: In C or Java, a variable is like a fixed plastic bucket. If you declare an `int` bucket, you can only ever put numbers in it. If you try to put a string in it, it won't fit. In Python, a variable is like a **sticky name tag**. You can take the `a` name tag off a number and slap it onto a string. The "type" lives on the object itself, not on the name tag.

Concrete Example FIRST: Because only the value carries a type, the *same* name may be rebound to a value of a completely different type during a program's life:

```
# Dynamic Binding
a = 5
print(a)
a = 'nitish'
print(a)
```

```
5
nitish
```

Plain-English: You can assign a number to `a`, and then on the very next line, assign a string to `a`. Python does not complain. It simply removes the `a` name tag from the integer `5` and attaches it to the string `'nitish'`.

Technical Definition: The name `a` first refers to an integer, then to a string. This freedom is **dynamic binding** — the binding between a variable name and an object (and therefore its type) can change dynamically at runtime.

The `None` keyword

`None` is Python's null value — a literal meaning "no value." Assigning it does not print anything by itself:

```
k = None
a = 5
b = 6
print('Program exe')
```

```
Program exe
```

Identifiers must exist before use

Referring to a name that was never assigned raises a `NameError`:

```
k
a = 5
b = 6
print('Program exe')
```

```
NameError                                Traceback (most recent call last)
----> 1 k
      2 a = 5
      3 b = 6
      4 print('Program exe')

NameError: name 'k' is not defined
```

Trap. Evaluating a bare name (`k`) that has not been bound is an error, *not* a silent no-op. Note also that execution stops at the failing line — the later assignments and the `print` never run.

Taking User Input

Prerequisites Block: * **Assumed Knowledge:** Variables, data types.

First-use gloss: `input()` reads a line typed by the user as a string.

`input()` reads a line typed by the user. Its optional argument is the prompt shown before the cursor.

```
# take input from users and store them in a variable
fnum = input('enter first number')
snum = input('enter second number')

# add the 2 variables
result = fnum + snum

# print the result
print(result)
```

```
enter first number45
enter second number56
4556
```

The result is surprising only until you notice the types. `45 + 56` would be `101`, but the output is `4556` — the two inputs were *concatenated* as text, not added as numbers.

`input()` always returns a string

```
fnum = input('enter first number')
snum = input('enter second number')
print(type(fnum), type(snum))
result = fnum + snum
print(result)
```

```
enter first number56
enter second number67
<class 'str'> <class 'str'>
5667
```

`type(...)` confirms both variables are `str`. For strings, the `+` operator means concatenation, so `'56' + '67'` is `'5667'`.

Trap. `input()` always returns a `str`, even when the user types digits. To do arithmetic on typed numbers you must convert them first — the subject of the next section.

Type Conversion

Prerequisites Block: * **Assumed Knowledge:** Variables, input, data types.

First-use gloss: Type Conversion is the process of changing a value from one data type to another.

Converting a value from one type to another is done by calling the target type's constructor: `int(...)`, `float(...)`, `str(...)`, and so on.

Why conversion is needed: the `int + str` error

Mixing incompatible types in an arithmetic expression fails:

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Python will not guess how to add an integer and a string together; it raises a `TypeError` instead.

Explicit conversion truncates toward zero (float → int)

```
# Explicit
# str -> int
int(4.5)
```

4

Converting a float to an `int` discards the fractional part (it does not round) — `4.5` becomes `4`.

Converting input before the operation

Wrapping each value in `int(...)` turns the concatenation into real addition:

```
fnum = input('enter first number')
snum = input('enter second number')
# add the 2 variables
result = int(fnum) + int(snum)
# print the result
print(result)
print(type(fnum))
```

```
enter first number56
enter second number67
123
<class 'str'>
```

Now `56 + 67` correctly gives `123`. But look at the last line: `fnum` is **still** a `str`.

Trap. `int(fnum)` returns a *new* integer value; it does not change `fnum`. The original variable keeps its string type unless you reassign it.

Converting at the point of input

The cleaner pattern converts as the value is read, so the variable holds the right type from the start:

```
fnum = int(input('enter first number'))
snum = int(input('enter second number'))
result = fnum + snum
print(result)
print(type(fnum))
```

```
enter first number56
enter second number67
123
<class 'int'>
```

Here `fnum` is genuinely an `int`.

Implicit conversion: booleans behave as numbers

Because `bool` is a subtype of `int`, booleans participate in arithmetic with `True` counting as 1 and `False` as 0:

```
a = True + 4
b = False + 10
print("a:", a)
print("b:", b)
```

```
a: 5
b: 10
```

Literals

Prerequisites Block: * Assumed Knowledge: Data types.

First-use gloss: A **literal** is a fixed value written directly in source code.

A *literal* is a fixed value written directly in source code. Python provides distinct notations for integers in several bases, for floats, for complex numbers, and for strings.

Integer literals in different bases

```
a = 0b1010    # Binary Literal
b = 100        # Decimal Literal
c = 0o310      # Octal Literal
d = 0x12c      # Hexadecimal Literal
```

Prefix	Base	Literal	Decimal value
0b	2	0b1010	10

Prefix	Base	Literal	Decimal value
(none)	10	100	100
0o	8	0o310	200
0x	16	0x12c	300

```
print(a, b, c, d)
```

```
10 100 200 300
```

Regardless of the notation used to *write* them, all four are ordinary integers once stored; printing shows their decimal values.

Float literals and scientific notation

```
# Float Literal
float_1 = 10.5
float_2 = 1.5e2      # 1.5 * 10**2
float_3 = 1.5e-3     # 1.5 * 10**-3
```

The `e` notation means “times ten to the power of.” So `1.5e2` is $1.5 \times 10^2 = 150.0$ and `1.5e-3` is $1.5 \times 10^{-3} = 0.0015$.

```
print(float_1, float_2, float_3)
```

```
10.5 150.0 0.0015
```

A standalone example of the same notation:

```
print(1.5e3)
```

```
1500.0
```

Complex literals

The imaginary unit is written as a `j` suffix. A complex number exposes `.real` and `.imag` attributes:

```
# Complex Literal
x = 3.14j
print(x, x.imag, x.real)
```

```
3.14j 3.14 0.0
```

String literals

Strings can be written several ways, and a few special prefixes change how the text is interpreted.

```
string      = 'This is Python'
strings     = "This is Python"
char        = "C"
multiline_str = """This is a multiline string with more than one line
code."""
unicode     = u"\U0001F606\U0001F606\U0001F923"
raw_str     = r"raw \n string"

print(string)
print(strings)
print(char)
print(multiline_str)
print(unicode)
print(raw_str)
```

```
This is Python
This is Python
C
This is a multiline string with more than one line code.
😂😂😂
raw \n string
```

The behaviours on display:

- **Single vs. double quotes** (`'...'` , `"..."`) are equivalent; choose whichever avoids escaping.
- **Triple quotes** (`"""..."""` or `'''...'''`) create multi-line strings.
- `u"..."` marks a Unicode string; `\U0001F606` is a Unicode escape naming a character by code point, here producing emoji.
- `r"..."` marks a *raw* string: backslash escapes are taken literally, so `\n` stays the two characters backslash-n instead of becoming a newline.

Key point. A raw string (`r"..."`) is invaluable for things like Windows paths and regular-expression patterns, where literal backslashes are wanted.

Boolean and `None` literals

`True` , `False` , and `None` are themselves literals — the only values of their respective kinds.

Operators

Prerequisites Block: * **Assumed Knowledge:** Variables, basic data types.

First-use gloss: Operators are special symbols that perform computations on values and variables.

Python's operators group into arithmetic, relational, logical, bitwise, assignment, and membership families.

Arithmetic operators

```
print(5 + 6)
print(5 - 6)
print(5 * 6)
print(5 / 2)
print(5 // 2)
print(5 % 2)
print(5 ** 2)
```

```
11
-1
30
2.5
2
1
25
```

Operator	Meaning	Example	Result
+	Addition	5 + 6	11
-	Subtraction	5 - 6	-1
*	Multiplication	5 * 6	30
/	True division	5 / 2	2.5
//	Floor division	5 // 2	2
%	Modulus (remainder)	5 % 2	1
**	Exponentiation	5 ** 2	25

Key point. `/` always produces a `float` (`5 / 2` is `2.5`, never `2`), while `//` performs floor division and keeps an integer result when both operands are integers. `%` returns the remainder, and `**` raises to a power.

Relational operators

Relational (comparison) operators compare two values and evaluate to a boolean. In the notebook they produce results such as `True`, `True`, `False`.

Operator	Meaning
<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal to
<code><=</code>	Less than or equal to

Logical operators

```
print(1 and 0)
print(1 or 0)
print(not 1)
```

```
0
1
False
```

`and` returns the first falsy operand (or the last value if all are truthy); `or` returns the first truthy operand; `not` inverts truthiness to a boolean. Here `1` is truthy and `0` is falsy, so `1 and 0` yields `0`, `1 or 0` yields `1`, and `not 1` yields `False`.

Bitwise operators

These act on the binary representations of integers.

```
print(2 & 3)    # bitwise and
print(2 | 3)    # bitwise or
print(2 ^ 3)    # bitwise xor
print(~3)
print(4 >> 2)
print(5 << 2)
```

2
3
1
-4
1
20

Operator	Name	Example	Result	Reasoning
&	AND	2 & 3	2	10 & 11 = 10
\	OR	2 \ 3	3	10 \ 11 = 11
^	XOR	2 ^ 3	1	10 ^ 11 = 01
~	NOT (invert)	~3	-4	~x = -(x + 1)
>>	Right shift	4 >> 2	1	100 shifted right twice → 1
<<	Left shift	5 << 2	20	101 shifted left twice → 10100

Key point. Shifting left by n multiplies by 2^{**n} ; shifting right by n divides by 2^{**n} (floor). Bitwise NOT of x is $-(x + 1)$, which is why ~ 3 is -4 .

Assignment operators

`=` binds a value to a name. The *augmented* assignment operators combine an operation with assignment.

```
# Assignment Operators
# =
# a = 2
a = 2

# a = a + 2
a += 2
print(a)
```

4

`a += 2` is shorthand for `a = a + 2`. The same pattern exists for the other arithmetic operators (`-=`, `*=`, `/=`, `//=`, `%=`, `**=`, and the bitwise ones).

Trap. Python has **no** `++` or `--` operators. The increment idioms `a++` and `++a` from C-style languages are not valid Python; use `a += 1` instead.

Membership operators

`in` and `not in` test whether a value occurs within a sequence (a string, list, tuple, and so on), returning a boolean.

```
print('D' in 'Delhi')
```

True

```
print('D' not in 'Delhi')
print(1 in [2, 3, 4, 5, 6])
```

False
False

'D' is a character of 'Delhi', so `in` is True and `not in` is False. The integer 1 is absent from `[2, 3, 4, 5, 6]`, so that membership test is False.

Control Flow: Loops

Prerequisites Block: * **Assumed Knowledge:** Operators, Booleans, basic data types.

First-use gloss: A **loop** is a programming construct that repeats a block of code as long as a specified condition is met or for every item in a sequence.

The `for` loop and `range()`

`range(start, stop, step)` generates a sequence of integers from `start` up to (but not including) `stop`, advancing by `step`. A negative step counts downward.

```
# For loop demo
for i in range(10, 0, -1):
    print(i)
```

```
10
9
8
7
6
5
4
3
2
1
```

The loop runs from `10` down to `1`; `stop` is `0`, which is excluded.

Iterating directly over a sequence

A `for` loop can walk the elements of any iterable directly — no index needed.

Over a **string**, it yields one character at a time:

```
for i in 'Delhi':
    print(i)
```

```
D
e
l
h
i
```

Over a **list**:

```
for i in [1, 2, 3, 4, 5]:
    print(i)
```

```
1
2
3
4
5
```

Over a **tuple** — identical behaviour, different bracket:

```
for i in (1, 2, 3, 4, 5):
    print(i)
```

```
1
2
```

```
3
4
5
```

The `for...else` and `while...else` construct

Heavy. A control-flow paradigm unique to Python that breaks standard intuition from other languages.

Analogy: Imagine searching a dark room for your keys with a flashlight. You sweep the beam across the room (the loop). If you spot the keys, you stop searching immediately (`break`). If you sweep the entire room and never see them, you conclude they aren't there (the `else` block). The `else` only happens if the search naturally finishes without being interrupted.

Concrete Example FIRST: The classic use is searching: `break` out when the target is found; let `else` report failure if the search runs to completion.

Target present — the `break` fires, so `else` is skipped:

```
# Scenario A: the item is found (break triggers, else is skipped)
numbers = [1, 2, 3, 4, 5]

for num in numbers:
    if num == 3:
        print("Found the number 3!")
        break
else:
    print("Searched the whole list, but didn't find a 3.")
```

Target absent — the loop completes, so `else` runs:

```
numbers = [1, 2, 4, 5]

for num in numbers:
    if num == 3:
        print("Found the number 3!")
        break
else:
    print("Searched the whole list, but didn't find a 3.")

# Output: "Searched the whole list, but didn't find a 3."
```

Plain-English: The `else` block attached to a loop only executes if the loop finishes all its iterations naturally. If you exit the loop early using a `break` statement, Python skips the `else` block entirely. Many developers mentally rename `else` to `no_break` to remember this behavior.

Technical Definition: Python loops may carry an `else` clause. The `else` block runs **only if the iterable is exhausted** (for `for`) or **the condition becomes `False`** (for `while`). If the loop is terminated by a `break` statement, the `else` block is unconditionally skipped.

`while...else`

The same rule applies to `while`: if the condition naturally becomes `False`, the `else` runs; a `break` would skip it.

```
count = 3

while count > 0:
    print(f"Countdown: {count}")
    count -= 1
else:
    print("Countdown finished successfully!")

# Output:
# Countdown: 3
# Countdown: 2
# Countdown: 1
# Countdown finished successfully!
```

Worked example: printing primes with `for...else`

A nested loop puts the construct to work. The inner loop tries every potential divisor of `i`; if one divides evenly, `i` is composite and the inner loop `break`s. Only when no divisor is found does the inner `else` run and print `i` — exactly the numbers that are prime.

```
lower = int(input('enter lower range'))
upper = int(input('enter upper range'))

for i in range(lower, upper + 1):
    for j in range(2, i):
        if i % j == 0:
            break
    else:
        print(i)
```

```
enter lower range10
enter upper range100
11
```

```
13
17
19
...
```

The `else` is attached to the **inner** `for`, so it fires once per `i` that survived the divisor test — the prime numbers in the requested range.

Key point. Loop `else` pairs naturally with `break`. Read it as “if the loop never broke, then...”. Without a `break` inside, the `else` is equivalent to code placed after the loop and adds nothing.

Strings

Prerequisites Block: * **Assumed Knowledge:** Data types, loops, operators.

First-use gloss: A **string** is a sequence of characters used to represent text.

What a string is

First-use gloss: A string is a sequence of characters. In Python specifically, a string is a sequence of **Unicode** characters.

The topics that organise this section:

- Creating strings
- Accessing strings
- Adding characters to strings
- Editing strings
- Deleting strings
- Operations on strings
- String functions

(The sessions captured here develop creation, access by index, slicing, and immutability; the remaining topics are introduced as the roadmap for what follows.)

Creating strings

```
s = 'hello'
s = "hello"
# multiline strings
s = '''hello'''
s = """hello"""
s = str('hello')
print(s)
```

```
hello
```

There are five interchangeable ways shown to build the same string: single quotes, double quotes, triple single quotes, triple double quotes, and the `str()` constructor.

Quotes inside strings

Choosing the *other* quote style lets a quote character appear literally inside the text:

```
s = "it's raining outside"
print(s)
```

```
it's raining outside
```

Wrapping in double quotes allows the apostrophe in `it's` without escaping.

Accessing characters: indexing

Each character has a position. **Positive indexing** counts from the left starting at `0`; **negative indexing** counts from the right starting at `-1`.

For `s = 'hello world'`:

Character	h	e	l	l	o	(space)	w	o	r	l	d
Positive idx	0	1	2	3	4	5	6	7	8	9	10
Negative idx	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Negative indexing in action:

```
# Negative Indexing
s = 'hello world'
print(s[-1])
```

```
d
```

```
print(s[-3])
```

```
r
```

`s[-1]` is the last character, `s[-3]` the third from the end.

Trap. Indexing past the end of a string raises `IndexError: string index out of range`. There is no “off the end” sentinel; the position must actually exist.

Slicing

A slice extracts a substring with the form `s[start:stop:step]`. `start` is included, `stop` is excluded, and `step` is the stride (default `1`). Any of the three may be omitted.

For all examples, `s = 'hello world'`.

Basic slice

```
print(s[0:5])
```

```
hello
```

Indices `0` through `4` — `'hello'` (position `5`, the space, is excluded).

Omitting an endpoint

Leaving out `start` defaults it to the beginning; leaving out `stop` defaults it to the end. The intended example slices to the first three characters:

```
print(s[:3])
```

```
hel
```

Trap (notebook hazard). The captured cell for `print(s[:3])` displayed `llo world`. That output is **stale** — it was left over from a previous run when the code read `s[2:]` (which *does* give `'llo world'`). The current code `s[:3]` evaluates to `'hel'`. In Jupyter/Colab, editing a cell does **not** refresh its shown output until you re-execute it; never trust a result you have not just run.

Slicing with a step

A `step` of `2` takes every second character:

```
print(s[0:6:2])
```

```
hlo
```

Positions 0, 2, 4 → h, l, o.

Negative step — walking backwards

A negative step moves right-to-left, so `start` should be greater than `stop`.

```
print(s[6:0:-1])
```

```
w olle
```

From position 6 (w) down to position 1 (stop 0 excluded): w, space, o, l, l, e.

```
print(s[6:0:-2])
```

```
wol
```

Same downward range, stride 2: positions 6, 4, 2 → w, o, l.

Reversing a string

Omitting both endpoints with a step of -1 reverses the whole string — the idiomatic Python reversal:

```
print(s[::-1])
```

```
dlrow olleh
```

Slicing with negative indices

Endpoints may be negative too. Taking from the fifth-from-last to the second-to-last:

```
print(s[-5:-1])
```

```
worl
```

Positions -5 through -2 → w, o, r, l.

The last five characters — negative `start`, omitted `stop`:

```
print(s[-5:])
```

```
world
```

A negative-everything reverse of a tail section:

```
print(s[-1:-6:-1])
```

```
dlrow
```

From `-1` (`d`) leftwards to `-6` excluded: `d`, `l`, `r`, `o`, `w`.

Key point. Read a slice as “from `start`, up to but not including `stop`, stepping by `step`.” With a positive step you move left→right and need `start < stop`; with a negative step you move right→left and need `start > stop`. Mismatched direction yields an empty string rather than an error.

Strings are immutable

A string cannot be changed in place. Attempting to assign to one of its positions fails:

```
s = 'hello world'
s[0] = 'H'
# Python strings are immutable
```

```
TypeError                                Traceback (most recent call last)
----> 1 s = 'hello world'
      2 s[0] = 'H'

TypeError: 'str' object does not support item assignment
```

Trap. Strings are **immutable**. You cannot edit a character by index; `s[0] = 'H'` raises `TypeError: 'str' object does not support item assignment`. To “change” a string, build a new one (for example by slicing and concatenation) and rebind the name.

Deleting strings

The `del` keyword deletes a name binding entirely. Attempting to delete a part of a string (such as a slice) fails because strings are immutable.

```
s = 'hello world'
del s
print(s)
```

```
NameError: name 's' is not defined
```

Attempting to delete a slice:

```
s = 'hello world'
del s[-1:-5:2]
print(s)
```

```
TypeError                                Traceback (most recent call last)
----> 2 del s[-1:-5:2]

TypeError: 'str' object does not support item deletion
```

Operations on strings

Strings support concatenation and repetition using arithmetic operators, as well as lexicographical comparison using relational operators.

Arithmetic operations

The `+` operator concatenates two strings. The `*` operator repeats a string a given number of times.

```
print('delhi' + ' ' + 'mumbai')
```

```
delhi mumbai
```

```
print('delhi'*5)
```

```
delhidelhidelhidelhidelhi
```

```
print("*"*50)
```

```
*****
```

Relational operations

Relational operators (`==`, `!=`, `>`, `<`, `>=`, `<=`) compare strings lexicographically (based on alphabetical order / Unicode values).

```
print('delhi' != 'delhi')
```

> ****Trap (notebook hazard).**** The captured cell for `print(s[:3])` displayed `> 'llo world'`. That output is ****stale**** – it was left over from a previous run
> when the code read `s[2:]` (which **does** give `'llo world'`). The current code
> `s[:3]` evaluates to `'hel'`. In Jupyter/Colab, editing a cell does ****not****
> refresh its shown output until you re-execute it; never trust a result you
> have not just run.

Slicing with a step

A `'step'` of `'2'` takes every second character:

```
```python
print(s[0:6:2])
```

```
hlo
```

Positions `0, 2, 4` → `h, l, o`.

### Negative step — walking backwards

A negative step moves right-to-left, so `start` should be greater than `stop`.

```
print(s[6:0:-1])
```

```
w olle
```

From position `6` (`w`) down to position `1` (`stop 0` excluded): `w`, space, `o`, `l`, `l`, `e`.

```
print(s[6:0:-2])
```

```
wol
```

Same downward range, stride `2`: positions `6, 4, 2` → `w, o, l`.

### Reversing a string

Omitting both endpoints with a step of `-1` reverses the whole string — the idiomatic Python reversal:

```
print(s[::-1])
```

```
dlrow olleh
```

### Slicing with negative indices

Endpoints may be negative too. Taking from the fifth-from-last to the second-to-last:

```
print(s[-5:-1])
```

```
worl
```

Positions `-5` through `-2` → `w`, `o`, `r`, `l`.

The last five characters — negative `start`, omitted `stop`:

```
print(s[-5:])
```

```
world
```

A negative-everything reverse of a tail section:

```
print(s[-1:-6:-1])
```

```
dlrow
```

From `-1` (`d`) leftwards to `-6` excluded: `d`, `l`, `r`, `o`, `w`.

**Key point.** Read a slice as “from `start`, up to but not including `stop`, stepping by `step`.” With a positive step you move left→right and need `start < stop`; with a negative step you move right→left and need `start > stop`. Mismatched direction yields an empty string rather than an error.

### Strings are immutable

A string cannot be changed in place. Attempting to assign to one of its positions fails:

```
s = 'hello world'
s[0] = 'H'
Python strings are immutable
```

```
TypeError Traceback (most recent call last)
----> 1 s = 'hello world'
 2 s[0] = 'H'

TypeError: 'str' object does not support item assignment
```

**Trap.** Strings are **immutable**. You cannot edit a character by index; `s[0] = 'H'` raises `TypeError: 'str' object does not support item assignment`. To “change” a string, build a new one (for example by slicing and concatenation) and rebind the name.

## Deleting strings

The `del` keyword deletes a name binding entirely. Attempting to delete a part of a string (such as a slice) fails because strings are immutable.

```
s = 'hello world'
del s
print(s)
```

```
NameError: name 's' is not defined
```

Attempting to delete a slice:

```
s = 'hello world'
del s[-1:-5:2]
print(s)
```

```
TypeError Traceback (most recent call last)
----> 2 del s[-1:-5:2]

TypeError: 'str' object does not support item deletion
```

## Operations on strings

Strings support concatenation and repetition using arithmetic operators, as well as lexicographical comparison using relational operators.

### Arithmetic operations

The `+` operator concatenates two strings. The `*` operator repeats a string a given number of times.

```
print('delhi' + ' ' + 'mumbai')
```

```
delhi mumbai
```

```
print('delhi'*5)
```

```
delhidelhidelhidelhidelhi
```

```
print("*"*50)
```

```

```

### Relational operations

Relational operators (`==`, `!=`, `>`, `<`, `>=`, `<=`) compare strings lexicographically (based on alphabetical order / Unicode values).

```
print('delhi' != 'delhi')
```

```
False
```

```
print('mumbai' > 'pune')
lexicographically
```

```
False
```

### Logical operations

Strings can be evaluated as booleans. An empty string is falsy; a non-empty string is truthy. When using `and`, Python returns the second operand if the first is truthy. When using `or`, Python returns the first operand if it is truthy.

```
print('hello' and 'world')
```

```
world
```

```
print('hello' or 'world')
```

```
hello
```

```
print('' and 'world')
```

```
print(not 'hello')
```

False

### Loops on strings

Strings are iterable. You can loop through a string character by character.

```
for i in 'delhi':
 print('pune')
```

pune  
pune  
pune  
pune  
pune

### Membership operations

You can check if a substring exists within a string using the `in` and `not in` operators. These operations are case-sensitive.

```
print('D' not in 'Delhi')
```

False

### String functions

Python provides common built-in functions that work with strings, as well as string-specific methods.

#### Common functions: len, max, min, sorted

- `len(s)` : Returns the number of characters in the string.
- `max(s)` : Returns the character with the highest Unicode value.
- `min(s)` : Returns the character with the lowest Unicode value.
- `sorted(s)` : Returns a list of characters sorted by their Unicode values.

```
s = 'hello world'
print(len(s))
```

11

```
print(max(s))
```

```
w
```

```
print(min(s))
```

```
print(sorted(s))
```

```
[' ', 'd', 'e', 'h', 'l', 'l', 'l', 'o', 'o', 'r', 'w']
```

### Capitalize, Title, Upper, Lower, Swapcase

These methods return a new string with modified casing.

```
s = 'hello world'
print(s.capitalize())
```

```
Hello world
```

```
print(s.title())
```

```
Hello World
```

```
print(s.upper())
```

```
HELLO WORLD
```

### Count, Find, Index

- `count(substring)` : Returns the number of non-overlapping occurrences of substring.
- `find(substring)` : Returns the lowest index in the string where the substring is found. Returns `-1` if not found.
- `index(substring)` : Like `find()`, but raises a `ValueError` when the substring is not found.

```
print('my name is nitish'.count('i'))
```

```
3
```

```
print('my name is nitish'.find('x'))
```

-1

```
print('my name is nitish'.index('x'))
```

ValueError: substring not found

### Endswith and Startswith

These methods return boolean values ( True or False ) checking if the string starts or ends with a specified suffix/prefix.

```
print('my name is nitish'.endswith('sho'))
```

False

```
print('my name is nitish'.startswith('my'))
```

True

### String Formatting

The `format()` method formats the specified value(s) and inserts them inside the string's placeholder.

```
name = 'nitish'
gender = 'male'
print('Hi my name is {} and I am a {}'.format(name, gender))
```

Hi my name is nitish and I am a male

You can use index numbers `{0}` to be sure the arguments are placed in the correct placeholders:

```
print('Hi my name is {1} and I am a {0}'.format(gender, name))
```

Hi my name is nitish and I am a male

### String Validation Methods

- `isalnum()` : Returns True if all characters in the string are alphanumeric.

- `isalpha()` : Returns True if all characters in the string are in the alphabet.
- `isdigit()` : Returns True if all characters in the string are digits.
- `isidentifier()` : Returns True if the string is a valid identifier (e.g., variable name).

```
print('nitish123'.isalnum())
```

True

## Split and Join

- `split(separator)` : Splits the string into a list of strings using the given separator. Default separator is whitespace.
- `join(iterable)` : Takes all items in an iterable and joins them into one string. The string on which it is called is used as the separator.

```
print('hi my name is nitish'.split())
```

```
['hi', 'my', 'name', 'is', 'nitish']
```

```
print('hi my name is nitish'.split('i'))
```

```
['h', ' my name ', 's n', 't', 'sh']
```

```
print(" ".join(['hi', 'my', 'name', 'is', 'nitish']))
```

hi my name is nitish

## Replace

Replaces a specified phrase with another specified phrase.

```
print('hi my name is nitish'.replace('nitish', 'campusx'))
```

hi my name is campusx

## Strip

Removes leading and trailing whitespaces.

```
print('nitish '.strip())
```

nitish

## # Cross-Cutting Themes and Common Traps

A consolidated list of the gotchas demonstrated across these sessions, gathered here for quick revision.

```
| Trap / behaviour | Why it happens |
|:-----|:-----|
| `input()` returns text, so `'45' + '56'` gives `'4556'` | Input is always a `str`; `+` concatenates strings rather than adding |
| `int(fnum)` leaves `fnum` a `str` | Conversions return a *new* value; they do not mutate the original variable |
| `int(4.5)` gives `4` | Float-to-int conversion truncates toward zero (it does not round) |
| `1e309` gives `inf` | Float overflow – exceeds the representable float range (this is float, not `int`, behaviour) |
| `True + 4` gives `5` | `bool` is a subtype of `int`, so `True` is `1` and `False` is `0` |
| `a++` / `++a` are invalid | Python has no increment/decrement operators; use `a += 1` |
| A bare undefined name raises `NameError` | An identifier must be bound before it is referenced |
| `int + str` raises `TypeError` | Python will not coerce across these incompatible types |
| Indexing past the end raises `IndexError` | The requested position must actually exist |
| `s[0] = 'H'` raises `TypeError` | Strings are immutable and do not support item assignment |
| A notebook cell shows a result that disagrees with its code | The output is stale; editing a cell does not re-run it |
| A loop's `else` runs only when no `break` fired | Read it as `no_break` – it triggers when the loop finishes naturally |
```

## # Interview Questions

### 1. What is Python? What are the benefits of using Python?

Python is a high-level, interpreted, general-purpose programming language. Being a general-purpose language, it can be used to build almost any type of application with the right tools/libraries. Additionally, python supports objects, modules, threads, exception-handling, and automatic memory management which help in modelling real-world problems and building applications to solve these problems.

**\*\*Benefits of using Python:\*\***

- Python is a general-purpose programming language that has a simple, easy-to-learn syntax that emphasizes readability and therefore reduces the cost of program maintenance. Moreover, the language is capable of scripting, is completely open-source, and supports third-party packages encouraging modularity and code reuse.
- Its high-level data structures, combined with dynamic typing and dynamic binding, attract a huge community of developers for Rapid Application Development and deployment.

**### 2. What is a dynamically typed language?**

Before we understand a dynamically typed language, we should learn about what typing is. Typing refers to type-checking in programming languages. In a **\*\*strongly-typed\*\*** language, such as Python, ``"1" + 2`` will result in a type error since these languages don't allow for "type-coercion" (implicit conversion of data types). On the other hand, a **\*\*weakly-typed\*\*** language, such as Javascript, will simply output ``"12"`` as result.

Type-checking can be done at two stages:

- **\*\*Static\*\*** - Data Types are checked before execution.
- **\*\*Dynamic\*\*** - Data Types are checked during execution. Python is an interpreted language, executes each statement line by line and thus type-checking is done on the fly, during execution. Hence, Python is a Dynamically Typed Language.

**### 3. What is an Interpreted language?**

An Interpreted language executes its statements line by line. Languages such as Python, Javascript, R, PHP, and Ruby are prime examples of Interpreted languages. Programs written in an interpreted language runs directly from the source code, with no intermediary compilation step.

**### 4. What is PEP 8 and why is it important?**

PEP stands for **\*\*Python Enhancement Proposal\*\***. A PEP is an official design document providing information to the Python community, or describing a new feature for Python or its processes. PEP 8 is especially important since it documents the style guidelines for Python Code. Apparently contributing to the Python open-source community requires you to follow these style guidelines sincerely and strictly.

**### 5. What are the common built-in data types in Python?**

There are several built-in data types in Python. Although, Python doesn't require data types to be defined explicitly during variable declarations type errors are likely to occur if the knowledge of data types and their

compatibility with each other are neglected. Python provides ``type()`` and ``isinstance()`` functions to check the type of these variables. These data types can be grouped into the following categories:

1. **\*\*None Type\*\***: None keyword represents the null values in Python. Boolean equality operation can be performed using these NoneType objects.
2. **\*\*Numeric Type\*\***: There are three distinct numeric types - integers, floating-point numbers and complex numbers. Additionally, booleans are a sub-type of integers.
3. **\*\*Sequence Types\*\***: According to Python Docs, there are three basic Sequence Types - lists, tuples, and range objects. Sequence types have the in and not in operators defined for their traversing their elements. These operators share the same priority as the comparison operations.
4. **\*\*Mapping Types\*\***: A mapping object can map hashable values to random objects in Python. Mappings objects are mutable and there is currently only one standard mapping type, the dictionary.
5. **\*\*Set Types\*\***: Currently, Python has two built-in set types - set and frozenset. set type is mutable and supports methods like ``add()`` and ``remove()``. frozenset type is immutable and can't be modified after creation.
6. **\*\*Callable Types\*\***: Callable types are the types to which function call can be applied. They can be user-defined functions, instance methods, generator functions, and some other built-in functions, methods and classes.

### ### 6. Operator Precedence

The operator precedence in Python is listed in the following table (descending order, upper group has higher precedence):

Operators	Meaning
<code>:`(``,`)`</code>	Parentheses
<code>`**`</code>	Exponent
<code>`+x``,`-x``,`~x`</code>	Unary plus, Unary minus, Bitwise NOT
<code>`*`,`/`,`//`,`%`</code>	Multiplication, Division, Floor division, Modulus
<code>`+`,`-`</code>	Addition, Subtraction
<code>`&lt;&lt;`,`&gt;&gt;`</code>	Bitwise shift operators
<code>`&amp;`</code>	Bitwise AND
<code>`^`</code>	Bitwise XOR
<code>` `</code>	Bitwise OR
<code>`==`,`!=`,`&gt;`,`&gt;=`,`&lt;`,`&lt;=`,`is`,`is not`,`in`,`not in`</code>	Comparisons, Identity, Membership operators
<code>`not`</code>	Logical NOT
<code>`and`</code>	Logical AND
<code>`or`</code>	Logical OR

### ### 7. Explain the ternary operator in Python.

Unlike C++, we don't have `?:` in Python, but we have this:  
`[on true] if [expression] else [on false]`

If the expression is True, the statement under `[on true]` is executed. Else, that under `[on false]` is executed.  
Below is how you would use it:

```
```python
a, b = 2, 3
min = a if a < b else b
print(min)
```

Above will print 2.

8. What Does the 'is' Operator Do?

Identity operators in Python, `is` and `is not`, are used to check if two values are located on the same part of the memory. Two variables that are equal does not imply that they are identical. `==` checks for equality, while `is` checks for identity (memory address).

For example:

```
a = 257
b = 257
print(a is b) # False
print(a == b) # True
```

Python caches small integers in the range -5 to 256, so if `a = 256` and `b = 256`, `a is b` would be `True`.

9. Disadvantages of Python.

Python has several disadvantages: - **Speed:** Python is an interpreted language and is slow compared to C/C++ or Java. It is a high-level language and execution takes place with the help of an interpreter instead of a compiler. - **Mobile Development:** Python is strong in desktop and server platforms, but it is considered a weak language for mobile development. Very few mobile applications are built in it. - **Memory Consumption:** For memory-intensive tasks, Python is not a good choice. Its memory consumption is high due to the flexibility of the data types. - **Database Access:** Python's database access layer is underdeveloped and primitive compared to popular technologies like JDBC and ODBC. This acts as a major barrier for big enterprises. - **Runtime Errors:** Python is dynamically typed, so it has errors that only show up at runtime. It requires more testing. - **Difficulty in Using Other Languages:** Python lovers become so accustomed to its features and extensive libraries that they face problems learning or working on other programming languages.

10. How strings are stored in Python?

Strings in Python are arrays of bytes representing Unicode characters. Python does not have a character data type; a single character is simply a string with a length of 1. Strings are immutable, meaning they cannot be changed after they are created.

11. What is the Zen of Python?

The Zen of Python is a collection of 19 “guiding principles” for writing computer programs that influence the design of the Python programming language. Some of the key principles include: - Beautiful is better than ugly. - Explicit is better than implicit. - Simple is better than complex. - Complex is better than complicated. - Flat is better than nested. - Sparse is better than dense. - Readability counts. - Special cases aren’t special enough to break the rules.

12. Identity operator (`is`) vs Equality operator (`==`)?

- **Identity operators:** The `is` and `is not` keywords are called identity operators that compare objects based on their identity (their memory location).
- **Equality operators:** The `==` and `!=` are called equality operators that compare objects based on their values.

13. Modules vs Packages vs Library in Python

Python uses some terms that you may not be familiar with if you are coming from a different language: - **Module:** A module is a Python file intended to be imported into scripts or other modules. It often defines classes, functions, and variables. - **Package:** A package is a collection of related modules that work together to provide certain functionality. These modules are contained within a folder and can be imported. This folder will often contain a special `__init__.py` file. - **Library:** A library is an umbrella term that loosely means “a bundle of code.” Libraries can have tens or hundreds of individual modules. The Python Standard Library contains hundreds of modules bundled with Python.

14. Why `0.3 - 0.2` is not equal to `0.1` in Python?

The reason behind it is called “precision,” and it’s due to the fact that computers do not compute in Decimal, but in Binary. Computers use a base-2 system (Binary code) to store floating-point numbers, which cannot represent numbers like `0.1` perfectly, leading to minor floating-point inaccuracies.

15. What are Python Docstrings?

Python docstrings are the string literals that appear right after the definition of a function, method, class, or module. They are used to document the code and explain its purpose and usage. They can be accessed using the `__doc__` attribute of the object.

Lists in Python

Prerequisites Block: * **Assumed Knowledge:** Variables, loops, basic types.

What are Lists?

A List is a data type where you can store multiple items under one name. More technically, lists act like **dynamic arrays**, which means you can add more items on the fly.

```
L = [20, 'Jessa', 35.75, [30, 60, 90]]
```

Lists can contain heterogeneous data types, including other lists.

Array vs. Lists

Feature	Arrays	Lists
Size	Fixed size	Dynamic size
Data Types	Homogeneous (same data type)	Heterogeneous (mixed data types)
Speed of Execution	Faster	Slower
Memory	Contiguous memory allocation	Referential arrays

How Lists are Stored in Memory

Heavy. Python lists are not traditional arrays; they are implemented as Referential Arrays.

Analogy: Think of a traditional C array as a row of lockers where the actual items (like books) are stuffed directly inside the lockers. A Python list, however, is a row of lockers that only contains **maps** (memory addresses) pointing to where the books are actually stored across a giant library.

Concrete Example FIRST: When you create a list `L = [1, 2, 3]`, Python creates objects in memory for the integers `1`, `2`, and `3`. The list `L` itself is an array of references (pointers) to those memory locations.

```
L = [1, 2, 3]
print(id(L[0]) == id(1))
```

True

Plain-English: Because Python caches small integers, the memory address of the first element in the list `id(L[0])` is exactly the same as the memory address of the integer `1` itself. The list doesn't "contain" the number `1`; it contains the address of `1`.

Technical Definition: In languages like C or Java, arrays are stored in contiguous memory locations. For example, an array of 3 integers will occupy 12 bytes consecutively. In Python, lists are implemented as **Referential Arrays**. The items in the list hold the memory addresses of the objects, not the values directly. This design is what allows lists to contain heterogeneous data types—since the list only stores memory addresses (which have a fixed size), it doesn't matter if one address points to an integer and another points to a massive string.

How Dynamic Arrays Work

When a list (which is a dynamic array) runs out of space to store new items, Python does the following: 1. It allocates a new, larger block of memory (usually double the size of the current array). 2. It copies all the existing items (references) from the old array to the new array. 3. It adds the new item. 4. The old array is deleted.

Characteristics of a List

1. **Ordered:** Lists maintain the order of elements as they were inserted. `[1, 2, 3]` is not the same as `[3, 2, 1]`.
2. **Changeable/Mutable:** You can modify, add, or remove elements after the list has been created.
3. **Heterogeneous:** Lists can contain elements of different data types (e.g., integers, strings, other lists).
4. **Can have duplicates:** Lists can contain multiple identical values.
5. **Dynamic:** Lists automatically resize themselves when elements are added or removed.
6. **Can be nested:** A list can contain another list as an item.
7. **Items can be accessed:** Elements can be accessed via indexing and slicing.
8. **Can contain any kind of object:** Any Python object can be stored in a list.

Creating a List

Lists can be created in several ways, and they can hold different numbers of dimensions or types of data.

```
# Empty List
print([])

# 1D List -> Homogeneous
print([1, 2, 3, 4, 5])
```

```
# 2D List
print([1, 2, 3, [4, 5]])

# 3D List
print([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

# Heterogeneous List
print([1, True, 5.6, 5+6j, 'Hello'])

# Using Type conversion (list() function)
print(list('hello')) # Output: ['h', 'e', 'l', 'l', 'o']
```

Accessing Items from a List

You can access items using index positions (0-based indexing) or slicing.

```
# Indexing
L = [[1, 2], [3, 4], [5, 6], [7, 8]]
# Accessing '2'
print(L[0][0][1])

## Adding Items to a List
There are three main ways to add items to a list:
```

1. `list.append()`: Adds a single item to the end of the `list`. If you pass a `list`, it gets added as a single nested `list`.
2. `list.extend()`: Adds multiple `items` (from an iterable) to the end of the `list` individually.
3. `list.insert()`: Adds a single item at a specific index.

```
```python
L = [1, 2, 3, 4, 5]

Using append()
L.append([6, 7, 8])
print(L) # Output: [1, 2, 3, 4, 5, [6, 7, 8]]

Using extend()
L = [1, 2, 3, 4, 5]
L.extend([6, 7, 8])
print(L) # Output: [1, 2, 3, 4, 5, 6, 7, 8]

extend() with a string adds individual characters
L.extend('delhi')
print(L) # Output: [..., 'd', 'e', 'l', 'h', 'i']

Using insert(index, item)
```

```
L = [1, 2, 3, 4, 5]
L.insert(1, 100) # Inserts 100 at index 1
print(L) # Output: [1, 100, 2, 3, 4, 5]
```

## Editing Items in a List

Lists are mutable, meaning their contents can be changed by assigning new values to specific indexes or slices.

```
L = [1, 2, 3, 4, 5]

Editing a single item via indexing
L[-1] = 500
print(L) # Output: [1, 2, 3, 4, 500]

Editing multiple items via slicing
L[1:4] = [200, 300, 400]
print(L) # Output: [1, 200, 300, 400, 500]
```

## Deleting Items from a List

There are several methods/keywords to remove items from a list.

1. **del keyword**: Can delete a single item by index, a slice, or the entire list variable.
2. **remove()**: Removes the first occurrence of a specific value.
3. **pop()**: Removes the item at the specified index (or the last item if no index is given) and returns it.
4. **clear()**: Empties the list, leaving an empty list `[]`.

```
L = [1, 2, 3, 4, 5]

Using del
del L[-1] # Deletes the last item
del L[1:3] # Deletes items from index 1 to 2
del L # Deletes the entire list variable from memory

Using remove()
L = [1, 2, 3, 4, 5]
Operations on Lists
Lists support various operations like arithmetic, membership, and looping.

Arithmetic Operations
- Concatenation (+): Merges two lists together.
- Repetition (*): Repeats the items in a list a specified number of times.

```python
```

```

L1 = [1, 2, 3, 4]
L2 = [5, 6, 7, 8]

# Concatenation
print(L1 + L2) # Output: [1, 2, 3, 4, 5, 6, 7, 8]

# Repetition
print(L1 * 3)  # Output: [1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4]

```

Membership Operations

You can use `in` and `not in` operators to check if an item exists within a list.

```

L1 = [1, 2, 3, 4, 5]
L2 = [1, 2, 3, 4, [5, 6]]

print(5 not in L1)      # Output: False
print([5, 6] in L2)     # Output: True
print(5 in L2)          # Output: False (5 is inside a nested list)

```

Loops

You can iterate through a list using a `for` loop.

```

L1 = [1, 2, 3, 4, 5]
for i in L1:
    print(i)

# Looping over nested lists
## Built-in Functions with Lists

```

Python provides several built-in functions to work with lists:

- `len()`: Returns the number of items in the list.
- `min()`: Returns the smallest item.
- `max()`: Returns the largest item.
- `sorted()`: Returns a new sorted list (does not modify original).

```

```python
L = [2, 1, 5, 7, 0]
print(len(L)) # Output: 5
print(min(L)) # Output: 0
print(max(L)) # Output: 7
print(sorted(L)) # Output: [0, 1, 2, 5, 7]

```

## List Methods

In addition to the adding and deleting methods, lists have:

- `count()` : Returns the number of times a value appears.
- `index()` : Returns the index of the first occurrence of a value.
- `reverse()` : Reverses the list *in place*.
- `sort()` : Sorts the list *in place* (modifies original).
- `copy()` : Returns a shallow copy of the list.

```
L = [1, 2, 1, 3, 4, 1, 5]
print(L.count(1)) # Output: 3
print(L.index(3)) # Output: 3

L = [2, 1, 5, 7, 0]
L.reverse()
print(L) # Output: [0, 7, 5, 1, 2]

L.sort()
print(L) # Output: [0, 1, 2, 5, 7]

L1 = L.copy() # Creates a shallow copy
```

## List Comprehension

**Heavy.** A syntactically compressed loop for generating lists.

**Analogy:** List comprehension is like placing a bulk order at a restaurant. Instead of going to the counter 10 times to say “I want a burger” (a `for` loop), you hand them a single piece of paper saying “I want 10 burgers, but only if they have cheese” (list comprehension). It does the exact same work but much faster and in one statement.

**Concrete Example FIRST: Example: Add numbers 1 to 10 to a list** Using a traditional loop:

```
L = []
for i in range(1, 11):
 L.append(i)
print(L) # Output: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Using list comprehension:

```
L = [i for i in range(1, 11)]
print(L) # Output: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

**Plain-English:** List comprehensions allow you to create a new list by looping over an existing iterable and applying an expression or filter to it, all squeezed into a single line

surrounded by brackets. It replaces the common pattern of creating an empty list and calling `.append()` inside a `for` loop.

**Technical Definition:** List comprehension provides a concise way to create lists based on existing iterables. It is more time-efficient and space-efficient than loops because it is optimized at the C-level in Python's implementation.

**Syntax:**

```
newlist = [expression for item in iterable if condition == True]
```

**Advantages:** - More time-efficient and space-efficient than loops. - Requires fewer lines of code. **Example 1: Add numbers 1 to 10 to a list** Using a loop:

```
L = []
for i in range(1, 11):
 L.append(i)
print(L) # Output: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Using list comprehension: **Example 2: Scalar multiplication on a vector**

```
v = [2, 3, 4]
s = -3
print([s * i for i in v]) # Output: [-6, -9, -12]
```

**Example 3: Add squares**

```
L = [1, 2, 3, 4, 5]
print([i**2 for i in L]) # Output: [1, 4, 9, 16, 25]
```

**Example 4: Print all numbers divisible by 5 (1 to 50)**

```
print([i for i in range(1, 51) if i % 5 == 0])
Output: [5, 10, 15, 20, 25, 30, 35, 40, 45, 50]
```

**Example 5: Find words starting with a specific letter**

```
languages = ['java', 'python', 'php', 'c', 'javascript']
print([lang for lang in languages if lang.startswith('p')])
Example 6: Nested `if` with List Comprehension
```python
basket = ['apple', 'guava', 'cherry', 'banana']
my_fruits = ['apple', 'kiwi', 'grapes', 'banana']

# Add fruit from my_fruits if it exists in basket AND starts with 'a'
print([fruit for fruit in my_fruits if fruit in basket if
```

```
fruit.startswith('a'))  
# Output: ['apple']
```

Example 7: Print a (3, 3) matrix using Nested List Comprehension

```
matrix = [[i * j for i in range(1, 4)] for j in range(1, 4)]  
print(matrix)  
**Example 8: Cartesian Products using List Comprehension**  
```python  
L1 = [1, 2, 3, 4]
L2 = [5, 6, 7, 8]
print([i * j for i in L1 for j in L2])
```

### Traversing a List

There are two main ways to traverse a list: 1. **Itemwise**: Iterate directly over the items. `python L = [1, 2, 3, 4] for i in L: print(i)`  
2. **Indexwise**: Iterate using the range of indices. `python L = [1, 2, 3, 4] for i in range(len(L)): print(L[i])`

### Zip Function

The `zip()` function takes iterables, aggregates them in a tuple, and returns an iterator. It's useful for operating on multiple lists simultaneously.

```
L1 = [1, 2, 3, 4]
L2 = [-1, -2, -3, -4]

Add items of 2 lists indexwise
print([i + j for i, j in zip(L1, L2)])
Output: [0, 0, 0, 0]
```

### Disadvantages of Python Lists

While highly flexible, Python lists come with trade-offs: - **Slow execution**: Operations like arbitrary insertion/deletion are  $O(n)$ . - **Risky usage**: Lists are mutable, which can lead to unintended side-effects if not careful due to aliasing.

```
python a = [1, 2, 3] b = a # 'b' points to the same list as 'a'
a.append(4) print(a) # Output: [1, 2, 3, 4] print(b) # Output:
[1, 2, 3, 4] (b changed too!)
```

- **Eats up more memory**: Lists allocate extra space to allow for fast appends, and store references to objects rather than the raw data itself.

Note: Lists can even contain functions and classes!

```
L = [1, 2, print, type, input]
print(L)
Output: [1, 2, <built-in function print>, <class 'type'>, ...]
```

## Tuples

---

**Prerequisites Block:** \* **Assumed Knowledge:** Lists, immutability.

A tuple in Python is similar to a list. The difference between the two is that we cannot change the elements of a tuple once it is assigned, whereas we can change the elements of a list. In short, a tuple is an **immutable list**. A tuple cannot be changed in any way once it is created.

### Characteristics of Tuples

- **Ordered:** Elements have a defined order that will not change.
- **Unchangeable (Immutable):** You cannot add, remove, or modify elements after creation.
- **Allows Duplicates:** Like lists, tuples can contain duplicate values.

### Plan of Attack

- Creating a Tuple
- Accessing Items
- Editing Items
- Adding Items
- Deleting Items
- Operations on Tuples
- Tuple Functions

### Creating Tuples

```
Empty tuple
t1 = ()

Create a tuple with a single item
Without a trailing comma, Python treats it as a string
t2 = ('hello')
print(type(t2)) # Output: <class 'str'>

With a trailing comma, it becomes a tuple
t2 = ('hello',)
```

```

print(type(t2)) # Output: <class 'tuple'>

Homogeneous tuple
t3 = (1, 2, 3, 4)

Heterogeneous tuple
t4 = (1, 2.5, True, [1, 2, 3])

Nested tuple
t5 = (1, 2, 3, (4, 5))

Using type conversion
t6 = tuple('hello')
print(t6) # Output: ('h', 'e', 'l', 'l', 'o')

```

## Accessing Items

Accessing items in a tuple works exactly the same as in a list. - **Indexing:** `t[0]`, `t[-1]` - **Slicing:** `t[1:4]` - **Nested Indexing:** `t5[-1][0]`

## Editing Items

You **cannot** edit items in a tuple. Like strings, tuples are immutable.

```

t3 = (1, 2, 3, 4)
t3[0] = 100
TypeError: 'tuple' object does not support item assignment

```

## Adding Items

Since tuples are immutable, you **cannot** add new items (no `append`, `extend`, or `insert` equivalent exists for tuples).

## Deleting Items

You **cannot** delete individual items from a tuple.

```

t5 = (1, 2, 3, (4, 5))
del t5[-1]
TypeError: 'tuple' object doesn't support item deletion

```

However, you can delete the entire tuple using the `del` keyword.

```

del t5

```

## Operations on Tuples

Like strings and lists, tuples support basic operations.

- **Concatenation ( + ):** Joins tuples together.

```
t1 = (1, 2, 3, 4)
t2 = (5, 6, 7, 8)
print(t1 + t2) # Output: (1, 2, 3, 4, 5, 6, 7, 8)
```

- **Repetition ( \* )**: Repeats the tuple.

```
print(t1 * 3) # Output: (1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4)
```

- **Membership**: `in` and `not in`.
- **Iteration**: Using `for` loops. ## Tuple Functions Like lists, tuples support many built-in functions:
- **len(t)** : Returns the number of items.
- **sum(t)** : Returns the sum of all items.
- **min(t) / max(t)** : Returns the minimum/maximum item.
- **sorted(t)** : Returns a **list** of sorted items (does not modify the tuple, as it's immutable).
- **t.count(x)** : Returns the number of times `x` occurs in the tuple.
- **t.index(x)** : Returns the index of the first occurrence of `x`.

## Difference between Lists and Tuples

- **Syntax**: Lists use `[]`, tuples use `()`.
- **Mutability**: Lists are mutable, tuples are immutable.
- **Speed**: Tuples are generally faster to iterate over and create than lists.
- **Memory**: Tuples consume less memory.

```
import sys
L = list(range(1000))
T = tuple(range(1000))
print('List size', sys.getsizeof(L)) # e.g., 9120
print('Tuple size', sys.getsizeof(T)) # e.g., 8056
```

- **Built-in functionality**: Lists have more built-in methods (like `append`, `remove`) than tuples.
- **Error Prone**: Lists are more prone to errors due to mutability (e.g., aliasing issues). Tuples are safer.
- **Usability**: Tuples are typically used for heterogeneous data types, while lists are often used for homogeneous data.

## Immutability and Aliasing

Because tuples are immutable, they do not suffer from aliasing side-effects like lists do.

```
With Lists (Mutable)
a = [1, 2, 3]
b = a
a.append(4)
print(a) # Output: [1, 2, 3, 4]
print(b) # Output: [1, 2, 3, 4]

With Tuples (Immutable)
a = (1, 2, 3)
b = a
a = a + (4,)
print(a) # Output: (1, 2, 3, 4)
print(b) # Output: (1, 2, 3)
```

## Special Syntax

### Tuple Unpacking

You can unpack the elements of a tuple into multiple variables. The number of variables must match the number of items in the tuple.

```
a, b, c = (1, 2, 3)
print(a, b, c) # Output: 1 2 3

Error if variables do not match items
a, b = (1, 2, 3)
ValueError: too many values to unpack (expected 2)
```

You can use tuple unpacking for swapping variables intuitively:

```
a = 1
b = 2
a, b = b, a
print(a, b) # Output: 2 1
```

You can use the `*` operator to gather remaining items into a list:

```
a, b, *others = (1, 2, 3, 4)
print(a, b) # Output: 1 2
print(others) # Output: [3, 4]
```

### Zippping Tuples

You can zip two tuples together using the `zip()` function, which creates an iterator of tuples pairing items from each iterable.

```

a = (1, 2, 3, 4)
b = (5, 6, 7, 8)

Convert the zip object to a list to view it
print(list(zip(a, b)))
Output: [(1, 5), (2, 6), (3, 7), (4, 8)]

```

## Sets

**Prerequisites Block:** \* **Assumed Knowledge:** Lists, tuples, mutability.

### Introduction to Sets

A **set** is an unordered collection of items. Every set element is unique (no duplicates) and must be immutable (cannot be changed).

However, a set itself is **mutable**. We can add or remove items from it. Sets can also be used to perform mathematical set operations like union, intersection, symmetric difference, etc.

### Characteristics

- **Unordered:** Items have no index or specific order.
- **Mutable:** The set itself can be changed (items can be added or removed).
- **No Duplicates:** A set cannot contain duplicate elements.
- **Can't contain mutable data types:** Elements inside a set must be immutable (like integers, floats, strings, or tuples). You cannot have a list, dictionary, or another set inside a set.

### Creating Sets

You can create a set using curly braces `{}` or the `set()` function.

```

Empty set
Note: s = {} creates an empty dictionary, not a set!
s = set()
print(type(s)) # Output: <class 'set'>

1D Set
s1 = {1, 2, 3}
print(s1)

Homogeneous and Heterogeneous
s2 = {1, 'hello', 4.5, (1, 2, 3)} # Tuple is allowed (immutable)
print(s2)

```

```
Output: {1, 4.5, (1, 2, 3), 'hello'} (order may vary)

Sets cannot contain mutable items like lists or other sets
s3 = {1, 2, 3, [4, 5]} # TypeError: unhashable type: 'list'

Duplicates are not allowed
Python treats True as 1 and False as 0
s4 = {1, 'hello', 4.5, True}
print(s4)
Output: {1, 4.5, 'hello'} (True is considered a duplicate of 1 and
ignored)

Using type conversion (passing an iterable to set())
s5 = set([1, 2, 3, 3, 2, 1])
print(s5) # Output: {1, 2, 3}
```

## Accessing Items

Since sets are **unordered**, they do not support indexing or slicing.

```
s = {1, 2, 3, 4}
print(s[0]) # TypeError: 'set' object is not subscriptable
```

To access elements, you typically loop through the set or check for membership using the `in` keyword.

## Adding Items

You can add elements to a set using `add()` and `update()`.

### `add()`

Adds a single element to the set.

```
s = {1, 2, 3, 4}
s.add(5)
print(s) # Output: {1, 2, 3, 4, 5}
```

### `update()`

Adds multiple elements (from an iterable like a list, tuple, or another set) to the set.

```
s = {1, 2, 3, 4}
s.update([5, 6, 7])
print(s) # Output: {1, 2, 3, 4, 5, 6, 7}
```

## Deleting Items

There are several methods to remove items from a set:

### `discard(item)`

Removes the specified item. **Does not raise an error** if the item is not present.

```
s = {1, 2, 3, 4, 5}
s.discard(50) # No error
```

### `remove(item)`

Removes the specified item. **Raises a KeyError** if the item is not present.

```
s.remove(50) # KeyError: 50
```

### `pop()`

Removes and returns an arbitrary item from the set.

```
s = {1, 2, 3}
item = s.pop() # Removes a random item
```

### `clear()`

Empties the set completely.

```
s.clear()
print(s) # Output: set()
```

### `del`

Deletes the set object from memory entirely.

```
del s
```

## Set Operations

Sets are highly useful because they support mathematical set operations.

```
s1 = {1, 2, 3, 4, 5}
s2 = {4, 5, 6, 7, 8}

Union (|): All items from both sets
print(s1 | s2) # {1, 2, 3, 4, 5, 6, 7, 8}

Intersection (&): Common items in both sets
print(s1 & s2) # {4, 5}

Difference (-): Items in s1 but not in s2
print(s1 - s2) # {1, 2, 3}
print(s2 - s1) # {6, 7, 8}
```

```
Symmetric Difference (^): Items in either set, but NOT in both
print(s1 ^ s2) # {1, 2, 3, 6, 7, 8}
```

## Membership Test

You can test if an item exists in a set using `in` or `not in`. (This is much faster in sets than in lists!).

```
s = {1, 2, 3, 4, 5}
print(1 in s) # True
print(10 not in s) # True
```

## Iteration

You can loop through a set using a `for` loop.

```
for i in {1, 2, 3}:
 print(i)
```

## Set Functions

Similar to lists and tuples, sets support built-in functions: - `len(s)` : Returns the number of items. - `sum(s)` : Returns the sum of items. - `min(s)` / `max(s)` : Returns the minimum/maximum item. - `sorted(s)` : Returns a **sorted list** of the set's items.

```
s = {3, 1, 4, 5, 2, 7}
print(sorted(s)) # Output: [1, 2, 3, 4, 5, 7]
print(sorted(s, reverse=True)) # Output: [7, 5, 4, 3, 2, 1]
```

## Other Set Methods

Sets have several built-in methods that are equivalent to the mathematical operators:

### `union()` and `update()`

`s1.union(s2)` returns a new set with all items from both sets. `s1.update(s2)` modifies `s1` in-place, adding all items from `s2`.

### `intersection()` and `intersection_update()`

`s1.intersection(s2)` returns a new set with common items. `s1.intersection_update(s2)` modifies `s1` in-place to keep only common items.

### `difference()` and `difference_update()`

`s1.difference(s2)` returns a new set with items in `s1` but not in `s2`. `s1.difference_update(s2)` modifies `s1` in-place, removing items found in `s2`.

### `symmetric_difference()` and `symmetric_difference_update()`

`s1.symmetric_difference(s2)` returns a new set with items in either set but not both.

`s1.symmetric_difference_update(s2)` modifies `s1` in-place.

### Checking Set Relations

```
s1 = {1, 2, 3, 4}
s2 = {7, 8, 5, 6}

isdisjoint(): Returns True if two sets have a null intersection
print(s1.isdisjoint(s2)) # True

s1 = {1, 2, 3, 4, 5}
s2 = {3, 4, 5}

issuperset(): Returns True if s1 contains all items of s2
print(s1.issuperset(s2)) # True

issubset(): Returns True if all items of s2 are in s1
print(s2.issubset(s1)) # True
```

### `copy()`

Returns a shallow copy of the set.

```
s1 = {1, 2, 3}
s2 = s1.copy()
```

### Frozenset

A **frozenset** is just an **immutable version** of a Python set object. While elements of a set can be modified at any time, elements of the frozenset remain the same after creation.

Due to this, frozensets can be used as keys in Dictionary or as elements of another set.

### Creating Frozenset

```
fs = frozenset([1, 2, 3])
print(fs) # frozenset({1, 2, 3})
```

### What works and what does not

- **Works:** All read operations (union, intersection, difference, membership, length, iteration, etc.).
- **Doesn't work:** All write operations (add, remove, update, clear, pop, discard, etc.).

### When to use Frozenset?

Since sets cannot contain mutable items, you cannot create a 2D set (a set of sets):

```
s = {{1, 2}, {3, 4}} # TypeError: unhashable type: 'set'
```

However, you **can** create a set of frozensets because frozensets are immutable (and therefore hashable):

```
s = {frozenset([1, 2]), frozenset([3, 4])}
print(s) # Output: {frozenset({1, 2}), frozenset({3, 4})}
```

## Set Comprehension

Just like lists, sets also support comprehensions for concise creation.

```
Create a set of numbers greater than 5
s = {i for i in range(1, 11) if i > 5}
print(s) # Output: {6, 7, 8, 9, 10}

Create a set of squares
squares = {i**2 for i in range(1, 6)}
print(squares) # Output: {1, 4, 9, 16, 25}
```

## Dictionaries

**Prerequisites Block:** \* **Assumed Knowledge:** Sets, keys, mutability.

### Introduction to Dictionaries

Dictionary in Python is a collection of key-value pairs, used to store data values like a map. Unlike other data types that hold only a single value as an element, a dictionary stores two paired values.

In some languages, this is known as a **map** or **associative array**.

```
d = {'name': 'nitish', 'age': 33, 'gender': 'male'}
print(d)
```

### Characteristics

- **Mutable:** Dictionaries can be modified (items added, removed, or changed) after creation.
- **Indexing has no meaning:** Values are accessed via their keys, not by numerical index positions. ## Creating a Dictionary

You can create dictionaries in multiple ways:

```

1. Empty dictionary
d = {}
or
d = dict()

2. 1D dictionary
d1 = {'name': 'nitish', 'gender': 'male'}

3. With mixed keys (keys can be any immutable type)
d2 = {(1, 2, 3): 1, 'hello': 'world'}

4. 2D dictionary (nested dictionary, like JSON)
d3 = {
 'name': 'nitish',
 'college': 'bit',
 'sem': 4,
 'subjects': {
 'dsa': 50,
 'maths': 67,
 'english': 34
 }
}

5. Using dict() function with a sequence of tuples
d4 = dict([('name', 'nitish'), ('age', 32), (3, 3)])
Output: {'name': 'nitish', 'age': 32, 3: 3}

Duplicate keys (the later value overwrites the earlier one)
d5 = {'name': 'nitish', 'name': 'rahul'}
print(d5) # Output: {'name': 'rahul'}

Using a mutable type as a key raises a TypeError
d6 = {'name': 'nitish', [1, 2, 3]: 2} # TypeError: unhashable type: 'list'

```

## Accessing Items

You can access the values of a dictionary using the key inside square brackets `[]`, or using the `get()` method.

```

my_dict = {'name': 'Jack', 'age': 26}

Using square brackets
print(my_dict['age']) # Output: 26
print(my_dict['address']) # Raises KeyError

Using get() method
print(my_dict.get('age')) # Output: 26

```

```
print(my_dict.get('address')) # Output: None (does not raise error)

Accessing items in a 2D dictionary
d3 = {'subjects': {'maths': 67, 'english': 34}}
print(d3['subjects']['maths']) # Output: 67
```

## Adding Key-Value Pair

You can add a new key-value pair by simply assigning a value to a new key.

```
d = {'name': 'nitish', 'age': 32, 3: 3}
d['gender'] = 'male'
d['weight'] = 72
print(d)
Output: {'name': 'nitish', 'age': 32, 3: 3, 'gender': 'male', 'weight': 72}
```

## Editing Key-Value Pair

If the key already exists, assigning a value to it updates the existing value.

```
d = {'name': 'nitish', 'sem': 4}
d['sem'] = 5
print(d) # Output: {'name': 'nitish', 'sem': 5}
```

## Removing Key-Value Pair

Dictionaries offer several methods to remove items:

1. **pop(key)** : Removes the item with the specified key and returns its value.
2. **popitem()** : Removes the last inserted key-value pair (in Python 3.7+) and returns it as a tuple.
3. **del** : Removes the item with the specified key, or deletes the entire dictionary.
4. **clear()** : Empties the dictionary.

```
d = {'name': 'nitish', 'age': 32, 3: 3, 'gender': 'male', 'weight': 72}

pop()
d.pop(3) # Removes the key 3
print(d) # {'name': 'nitish', 'age': 32, 'gender': 'male', 'weight': 72}

popitem()
d.popitem() # Removes ('weight', 72)
print(d) # {'name': 'nitish', 'age': 32, 'gender': 'male'}

del
del d['age']
```

```
print(d) # {'name': 'nitish', 'gender': 'male'}
del d # Deletes the dictionary entirely

clear()
d.clear()
print(d) # {}
```

## Dictionary Operations

### Membership

The `in` and `not in` operators can check if a **key** exists in the dictionary. Membership tests are only for keys, not for values.

```
s = {'name': 'nitish', 'college': 'bit', 'sem': 5}

print('name' in s) # Output: True
print('nitish' in s) # Output: False ('nitish' is a value, not a key)
```

### Iteration

You can loop through a dictionary using a `for` loop. By default, it iterates over the keys.

```
d = {'name': 'nitish', 'gender': 'male', 'age': 33}

for i in d:
 print(i, d[i])
Output:
name nitish
gender male
age 33
```

## Dictionary Functions

### `len()`

Returns the number of key-value pairs in the dictionary.

```
print(len(d)) # Output: 3
```

### `sorted()`

Returns a list of all the **keys** in sorted order.

```
print(sorted(d)) # Output: ['age', 'gender', 'name']
```

`max()` / `min()`

Returns the maximum or minimum **key** in the dictionary.

`items()`, `keys()`, `values()`

These methods return view objects that provide a dynamic view of the dictionary's entries.

- `keys()` : Returns a view object displaying a list of all the keys.
- `values()` : Returns a view object displaying a list of all the values.
- `items()` : Returns a view object displaying a list of dictionary's (key, value) tuple pairs.

```
print(d.keys()) # Output: dict_keys(['name', 'gender', 'age'])
print(d.values()) # Output: dict_values(['nitish', 'male', 33])
print(d.items()) # Output: dict_items([('name', 'nitish'), ('gender',
'male'), ('age', 33)])
```

## Dictionary Comprehension

Similar to list and set comprehensions, you can create dictionaries using comprehension.

**Syntax:** `{key: value for vars in iterable}`

```
Print 1st 10 numbers and their squares
squares = {i: i**2 for i in range(1, 11)}
print(squares)
Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81, 10:
100}

Using an existing dictionary
distances = {'delhi': 1000, 'mumbai': 2000, 'bangalore': 3000}
distances_miles = {key: value * 0.62 for (key, value) in distances.items()}
print(distances_miles)
Output: {'delhi': 620.0, 'mumbai': 1240.0, 'bangalore': 1860.0}

Using zip
days = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday",
"Saturday"]
temp_C = [30.5, 32.6, 31.8, 33.4, 29.8, 30.2, 29.9]
temperatures = {i: j for (i, j) in zip(days, temp_C)}
print(temperatures)
Output: {'Sunday': 30.5, 'Monday': 32.6, ...}

Using if condition
products = {'phone': 10, 'laptop': 0, 'charger': 32, 'tablet': 0}
in_stock = {key: value for (key, value) in products.items() if value > 0}
print(in_stock)
Output: {'phone': 10, 'charger': 32}

Nested Comprehension
```

```
Print tables of numbers from 2 to 4
tables = {i: {j: i * j for j in range(1, 11)} for i in range(2, 5)}
print(tables)
Output:
{
2: {1: 2, 2: 4, 3: 6, ... 10: 20},
3: {1: 3, 2: 6, 3: 9, ... 10: 30},
4: {1: 4, 2: 8, 3: 12, ... 10: 40}
}
```

## Functions

**Prerequisites Block:** \* **Assumed Knowledge:** Loops, variables, data structures.

**First-use gloss:** A **function** is a reusable block of code that performs a specific task.

### Introduction to Functions

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing.

```
Syntax
def function_name(parameters):
 """docstring"""
 # body
 return expression
```

#### Example:

```
def is_even(i):
 """
 Input: i, a positive int
 Returns True if i is even, otherwise False
 """
 print("inside is_even")
 return i % 2 == 0
```

- **def** : Keyword used to declare a function.
- **is\_even** : The name of the function.

- **(i)** : Parameters or arguments passed to the function.
- **""" """** : Docstring, which provides documentation for the function.
- **Body**: The block of code inside the function.

You call the function later in the code using its name and passing the required arguments.

## Types of Arguments

When calling a function, you can pass arguments in different ways:

1. **Default Argument**
2. **Positional Argument**
3. **Keyword Argument**

```
def power(a=1, b=1):
 return a**b

Default arguments used (a=1, b=1)
print(power()) # Output: 1

Positional arguments (a=2, b=3)
print(power(2, 3)) # Output: 8

Keyword arguments (order doesn't matter)
print(power(b=3, a=2)) # Output: 8
```

### **\*args and \*\*kwargs**

**\*args** and **\*\*kwargs** are special Python keywords that are used to pass a variable length of arguments to a function.

#### **\*args**

**\*args** allows us to pass a variable number of **non-keyword arguments** (positional arguments) to a function. Inside the function, the arguments are collected into a tuple.

```
def multiply(*args):
 product = 1

 # args is a tuple of all passed arguments
 print(args)
 for i in args:
 product = product * i

 return product

print(multiply(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12))
Output:
```

```
(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12)
43545600
```

Note: `args` is just a naming convention. You can use any name, like `*salman` or `*numbers`, as long as it starts with an asterisk `*`.

### **\*\*kwargs**

`**kwargs` allows us to pass any number of **keyword arguments**. Inside the function, these arguments are collected into a dictionary.

```
def display(**kwargs):
 # kwargs is a dictionary
 for (key, value) in kwargs.items():
 print(key, '->', value)

display(india='delhi', srilanka='colombo', nepal='kathmandu',
 pakistan='islamabad')
Output:
india -> delhi
srilanka -> colombo
nepal -> kathmandu
pakistan -> islamabad
```

## Accessing Docstrings

You can access the docstring of a function using the `__doc__` attribute.

```
def is_even(i):
 """
 This function returns if a given number is odd or even
 input - any valid integer
 output - odd/even
 created on - 16th Nov 2022
 """
 return i % 2 == 0

print(is_even.__doc__)
```

### Points to remember while using `*args` and `**kwargs`

- **Order of arguments matters:** normal -> `*args` -> `**kwargs`
- The words `args` and `kwargs` are only a convention; you can use any name of your choice as long as they are preceded by `*` and `**` respectively.

## How Functions are Executed in Memory

When a function is called, the execution is managed using frames in memory:

1. **Global Frame:** When a function is defined, it is created and stored in the global frame.
2. **Local Frame:** When the function is called, a separate local frame is created for that specific function call.
3. **Variable Assignment:** Variables passed to the function are stored in the local frame.
4. **Return and Destruction:** Once the function finishes execution and returns a value, the local frame is destroyed, and control returns to the global frame.

#### Example:

```
def is_even(num):
 if num % 2 == 0:
 return 'even'
 else:
 return 'odd'

print(is_even(7))
```

- The function `is_even` is stored in the global frame.
- When `is_even(7)` is called, a local frame is created for `is_even`, and `num` is set to `7`.
- The function evaluates to `'odd'` and returns it.
- The local frame for `is_even(7)` is destroyed. ## Functions Without a Return Statement

If a function does not explicitly return a value using the `return` keyword, it implicitly returns `None`.

```
def is_even(num):
 if num % 2 == 0:
 print('even')
 else:
 print('odd')

Prints 'odd', then prints 'None' because is_even returns None
print(is_even(7))
```

This behavior is common with many built-in list methods that mutate the list in place instead of returning a new list:

```
L = [1, 2, 3]
print(L.append(4)) # Output: None
print(L) # Output: [1, 2, 3, 4]
```

## Variable Scope

Variables in Python have different scopes which determine where they can be accessed.

## Global vs. Local Variables

Variables defined outside of a function are **global** and can be accessed anywhere. Variables defined inside a function are **local** to that function.

```
def g(y):
 # Can read the global variable x
 print(x) # 5
 print(x + 1) # 6

x = 5
g(x)
print(x) # 5
```

If you assign a value to a variable inside a function, it creates a new local variable, shadowing the global one:

```
def f(y):
 x = 1 # Local variable x
 x += 1
 print(x) # 2

x = 5 # Global variable x
f(x)
print(x) # 5
```

## UnboundLocalError

If you try to modify a global variable inside a function without declaring it as global, Python assumes it's a local variable. This leads to an error if the variable hasn't been assigned yet.

```
def h(y):
 # Tries to access local 'x' before it is assigned
 x += 1

x = 5
h(x)
UnboundLocalError: local variable 'x' referenced before assignment
```

## The `global` Keyword

To modify a global variable inside a function, you must explicitly declare it using the `global` keyword.

```
x = 5

def h():
```

```
global x
x += 1

h()
print(x) # 6
```

## Modifying Mutable Global Objects

If your global variable is a mutable object (like a list, dictionary, or set), you **do not** need the `global` keyword to change its contents. This is because you are modifying the existing object in memory, not pointing the variable to a completely new object.

```
inventory = ["sword", "shield"]

def add_item(item):
 # No 'global' keyword needed
 inventory.append(item)

add_item("potion")
print(inventory) # ['sword', 'shield', 'potion']
```

**The Golden Rule:** If you need to use the equals sign (`=`) to assign a completely new value to a global variable name, use `global`. If you are just using methods to mutate an existing object in place, you don't need it.

## The “Sticky Note” Rule (Immutability)

In Python, variables are not buckets that hold data; they are just sticky notes (labels) attached to objects in memory.

When you create an integer, Python builds an integer object in memory and slaps your variable name on it like a sticky note. Because integers are **immutable**, their core value can never be changed.

```
x = 5
x = 6
```

When you do this, you didn't change the `5` into a `6`. Instead, Python: 1. Created an immutable object `5` in memory. 2. Put the sticky note `x` on the `5`. 3. Created a brand-new immutable object `6` in memory. 4. Ripped the sticky note `x` off the `5` and stuck it onto the `6`.

The original `5` was never altered. You can prove this using Python's built-in `id()` function, which reveals the exact memory address of an object.

## Nested Functions

Python allows you to define functions inside other functions. The inner function is only accessible from within the outer function (unless returned).

```
def f():
 def g():
 print('inside function g')
 g()
 print('inside function f')

f()
Output:
inside function g
inside function f
```

## Functions as First-Class Citizens

In Python, functions are “first-class citizens” (also known as first-class objects). This means that functions support all the operations generally available to other entities like integers, strings, and lists.

You can perform typical operations on functions such as: 1. Finding their type and memory address. 2. Reassigning them to new variables. 3. Deleting them. 4. Storing them in data structures (like lists or dictionaries). 5. Passing them as arguments to other functions. 6. Returning them from other functions.

### 1. Type and ID

```
def square(num):
 return num**2

2. Reassigning
```python
x = square
print(id(x)) # Same as id(square)
print(x(3)) # 9
```

3. Deleting

```
del square
square(3) # NameError: name 'square' is not defined
```

4. Storing

```
def square(num): return num**2
L = [1, 2, 3, 4, square]
print(L[-1](3)) # 9
```

```
s = {square}
print(s) # {<function __main__.square(num)>}
```

5. Returning a function

```
def f():
    def x(a, b):
        return a + b
    return x

val = f()(3, 4)
print(val) # 7
```

6. Passing a function as an argument

```
def func_a():
    print('inside func_a')

def func_b(z):
    print('inside func_c')
    return z()

print(func_b(func_a))
# Output:
# inside func_c
# inside func_a
# None
```

Lambda Functions

Heavy. Syntactically compressed anonymous functions.

Analogy: A lambda is like a disposable sticky note. While a normal `def` function is a formal, named recipe written in your permanent cookbook, a lambda is a quick calculation you scribble on a scrap of paper, use once, and throw away.

Example FIRST:

```
# x, y -> x+y
a = lambda x, y: x + y
print(a(5, 2)) # 7
```

Plain-English: A lambda function allows you to write a small, nameless function in a single line. It implicitly returns the result of its single expression without needing the `return` keyword.

Technical Definition: A lambda function is an anonymous inline function consisting of a single expression. Evaluating a lambda expression yields a function object. **Syntax:** `lambda arguments: expression` - The colon (`:`) separates the arguments from the expression.

Why here / The Trap: Why use them? They are primarily used as “throwaway” arguments for Higher-Order Functions (HOF). > **Trap.** Because they have no name and no `return` keyword, attempting to write multi-line logic or complex statements inside a lambda will fail. They must remain a single expression.

Higher Order Functions (HOF)

Heavy. Functions that treat other functions as data.

Analogy: If a normal function is a machine that processes raw materials (data) into products, a Higher-Order Function is a factory manager that takes in *other machines* (functions) and orchestrates them.

Example FIRST:

```
# Square the items of a list
L = [1, 2, 3, 4, 5]
list(map(lambda x: x**2, L))
# [1, 4, 9, 16, 25]
```

Plain-English: A Higher-Order Function is any function that either accepts another function as an argument, or returns a function as its result. They allow you to apply a generic behavior (like transforming or filtering) using a specific rule (the passed-in function).

Technical Definition: A higher-order function takes one or more function objects as arguments or returns a function object.

1. `map()`

Plain-English: It applies a given function to every item in an iterable, returning a map object (an iterator) with the transformed items. **Syntax:** `map(function, iterable)`

Examples:

```
# Odd/even labelling of list items
L = [1, 2, 3, 4, 5]
list(map(lambda x: 'even' if x % 2 == 0 else 'odd', L))
```

```
# ['odd', 'even', 'odd', 'even', 'odd']

# Fetch names from a list of dictionaries
users = [
    {'name': 'Rahul', 'age': 45, 'gender': 'male'},
    {'name': 'Nitish', 'age': 33, 'gender': 'male'},
    {'name': 'Ankita', 'age': 50, 'gender': 'female'}
]
list(map(lambda user: user['name'], users))
# ['Rahul', 'Nitish', 'Ankita']
```

2. filter()

Plain-English: It returns an iterator containing only the items from the original iterable that return `True` when passed into the given function. **Syntax:** `filter(function, iterable)`

Examples:

```
# Numbers greater than 5
L = [3, 4, 5, 6, 7]
list(filter(lambda x: x > 5, L))
# [6, 7]

# Fetch fruits starting with 'a'
fruits = ['apple', 'guava', 'cherry']
list(filter(lambda x: x.startswith('a'), fruits))
# ['apple']
```

3. reduce()

Plain-English: It applies a rolling computation to sequential pairs of values in an iterable, reducing the entire collection down to a single aggregate value. **Syntax:** `functools.reduce(function, iterable)`

Example:

```
import functools

# Sum of all items
L = [1, 2, 3, 4, 5]
functools.reduce(lambda x, y: x + y, L)
# 15
```

(Failure case: `reduce` requires `import functools` in Python 3; forgetting this will raise a `NameError`.)

Memory and Object References

Prerequisites Block: * **Assumed Knowledge:** Basic data types (lists, tuples, strings), variables and binding. * **Recap:** Variables in Python are sticky name tags bound to objects in memory, not fixed buckets.

Memory Models

When a variable is assigned a value, Python creates an object in memory and binds the variable name to that object's memory address. For example, with `a = 5`, an integer object `5` is created at some memory address, and `a` points to it.

Aliasing

First-use gloss: Aliasing occurs when multiple variable names refer to the exact same object in memory.

If we write:

```
a = [1, 2, 3]
b = a
```

Here, `b` is not a new copy of the list. Instead, `b` becomes an alias for `a`; both names point to the very same list object. Modifying the list through `b` (e.g., `b.append(4)`) will reflect in `a` because there is only one underlying object.

Garbage Collection

First-use gloss: Garbage Collection is Python's automatic memory management process that frees up memory occupied by objects that are no longer referenced by any variable.

Python uses reference counting. If an object's reference count drops to zero (meaning no variables point to it), the garbage collector reclaims that memory.

Mutability and Its Dangers

First-use gloss: Mutability refers to whether an object's internal state can be modified after creation. Lists and dictionaries are mutable; tuples, strings, and integers are immutable.

Mutability is powerful but dangerous in scenarios involving aliasing or when passing mutable objects to functions. Unintended side-effects occur if multiple parts of a program share a reference to a mutable object and one part modifies it, unexpectedly affecting the other parts.

Cloning (Shallow vs Deep Copies)

To avoid the dangers of aliasing with mutable objects, we can clone them.

Shallow Copy

Using `b = a[:]` or `b = a.copy()` creates a shallow copy. It creates a new list object, but inserts references to the *same* child objects found in the original list. * For a flat list like `[1, 2, 3]`, this is perfectly sufficient. Modifying `b` will not affect `a`. * For a nested list like `[1, 2, [3, 4]]`, a shallow copy only copies the outer list. The inner list `[3, 4]` is still shared between `a` and `b`.

Deep Copy

Using `import copy; copy.deepcopy(a)` creates a fully independent clone of the original object and all its nested objects recursively. Modifying deeply nested structures in the deep copy will not affect the original.

Memory Layout of Types

Nested Lists

A nested list like `[1, 2, [3, 4]]` is stored as an array of references. The first two elements point to integer objects `1` and `2`. The third element points to another list object, which in turn contains references to integer objects `3` and `4`.

Strings

Python strings are arrays of references to character objects. Like tuples, they are contiguous in memory.

Tuples vs. Lists in Memory

Heavy: Why do tuples take less memory than lists?

Analogy: A list is like a moving box with extra room left inside just in case you buy more stuff. A tuple is like shrink-wrapping the exact items you have.

Technical Definition: * **Lists** use dynamic arrays. When a list is created, Python over-allocates memory (reserves extra space) to allow for efficient future `append()` operations without needing to reallocate the whole array every time. * **Tuples** are static arrays. Because they are immutable, Python allocates exactly the amount of memory needed for the elements, with zero over-allocation. This makes tuples more memory-efficient than lists.

Sets and Dictionaries

Heavy: Why are mutable types not allowed as elements in Sets or keys in Dictionaries?

Sets and Dictionaries rely on hash tables for fast $O(1)$ lookups. The index of an item in the hash table is decided by calculating the **hash value** of the item. If mutable types (like lists) were allowed as keys, their content could change after insertion. If the content changes, the hash value would change, meaning the object would now be in the “wrong” bucket in the hash table, breaking the data structure and making it impossible to find the item again. Therefore, only immutable (hashable) types are permitted.

Object-Oriented Programming (OOP) Concepts

Prerequisites Block: * **Assumed Knowledge:** Functions, variables, data types.

Object-Oriented Programming is a paradigm that organizes software design around data, or objects, rather than functions and logic.

Class and Object

First-use gloss: A **Class** is a blueprint or template for creating objects. It defines the data (properties) and behaviors (functions/methods) that its objects will have. **First-use gloss:** An **Object** is a concrete instance of a Class.

Equation: `Class = Data (or property) + Functions (or behavior)`

For example, a `Car` class might have properties like `color` and `brand`, and behaviors like `accelerate()` and `brake()`. A specific red Toyota is an object (instance) of the `Car` class.

The Four Pillars of OOP

While the notes briefly touch on the terminology, the core concepts of OOP include: 1. **Encapsulation:** Grouping data and methods into a single unit (the class) and restricting direct access to some of the object's components. 2. **Abstraction:** Hiding complex implementation details and showing only the essential features of the object. 3. **Inheritance:** Allowing a new class to inherit properties and behaviors from an existing class. 4. **Poly-morphism:** The ability of different objects to respond to the same method call in their own specific ways.

File Handling

Prerequisites Block: * **Assumed Knowledge:** Strings, loops.

File handling allows Python programs to read from and write to external files on the disk, making data persistence possible.

Opening and Closing Files

The built-in `open()` function is used to access files. It returns a file object.

```
f = open('sample.txt', 'w')
f.write('Hello world')
f.close()
```

First-use gloss: `close()` flushes the in-memory buffers and releases the file back to the operating system. Attempting to write to or read from a closed file raises a `ValueError: I/O operation on closed file.`

File Modes

The `mode` argument tells Python what you intend to do with the file.

Character	Meaning
'r'	Read (default). Opens for reading. Errors if file doesn't exist.
'w'	Write . Opens for writing, truncating (overwriting) the file first.

Character	Meaning
'x'	Exclusive creation. Fails if the file already exists.
'a'	Append. Opens for writing, appending to the end of the file if it exists.
'b'	Binary mode. For reading/writing raw bytes (e.g., images).
't'	Text mode (default).

Overwriting vs. Appending

When using 'w' mode, if the file already exists, its entire contents are erased before writing the new data. To add new data without erasing existing content, use append mode 'a' :

```
f = open('sample.txt', 'a')
f.write('\nI am fine')
f.close()
```

Reading from Files

read()

read() reads the entire content of the file into a single string.

```
f = open('sample.txt', 'r')
s = f.read()
print(s)
f.close()
```

To read only a specific number of characters, pass the byte/character count as an argument. Subsequent calls pick up where the previous one left off:

```
f = open('sample.txt', 'r')
s = f.read(10) # reads the first 10 characters
print(s)
f.close()
```

readline()

readline() reads the file line by line. Each call returns the next line.

```
f = open('sample.txt', 'r')
print(f.readline())
```

```
print(f.readline())
f.close()
```

To read an entire file efficiently line by line:

```
f = open('sample.txt', 'r')
while True:
    data = f.readline()
    if data == '':
        break
    else:
        print(data, end='')
f.close()
```

writelines()

To write a list of strings directly into a file, use `writelines()` :

```
L = ['hello\n', 'hi\n', 'how are you\n', 'I am fine\n']
f = open('sample.txt', 'w')
f.writelines(L)
f.close()
```

The Context Manager (with block)

Heavy: Managing file closure manually is error-prone.

If a program crashes before reaching `f.close()` , the file might remain open, leading to resource leaks or corrupted data.

The modern, Pythonic way to handle files is using the `with` keyword, which acts as a **Context Manager**.

```
with open('sample.txt', 'w') as f:
    f.write('hello')
```

How it works: As soon as the indented block finishes (either normally or due to an error), the context manager automatically and safely closes the file. There is no need to write `f.close()` .

Reading Large Files in Chunks

When dealing with massive files (e.g., gigabytes in size), using `read()` will attempt to load the entire file into memory (RAM), which can crash the system.

Instead, read the file in manageable chunks using a loop:

```
with open('big.txt', 'r') as f:
    chunk_size = 100
    chunk = f.read(chunk_size)

    while len(chunk) > 0:
        print(chunk)
        chunk = f.read(chunk_size)
```

This guarantees that only 100 characters are kept in memory at any given time, regardless of how large the file is.